

RAW SOCKET

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심

MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.

<http://mmcn.ajou.ac.kr>



Contents

- ❖ Standard Sockets
- ❖ Raw Socket
- ❖ Example : ping
- ❖ IP_HDRINCL option



STANDARD SOCKETS

TCP/IP – Revisited

❖ TCP/IP Protocol Suites

OSI 7 Layers

Application
Presentation
Session
Transport
Network
Data Link
Physical

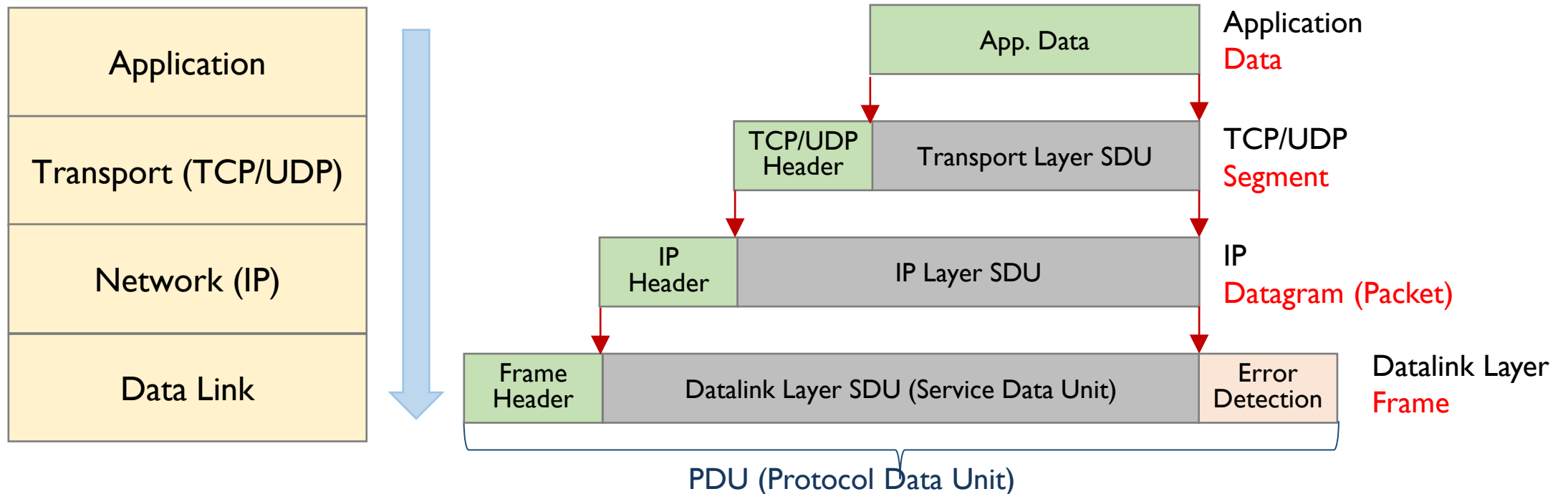
TCP/IP

Application		HTTP, SMTP, POP3, IMAP, FTP, DNS, RTP, RTSP,				
Transport		TCP, UDP				
Internet		IP (Internet Protocol)				
Network Access	Datalink	802.3 Ethernet	802.5 Token Ring	802.11 WLAN	802.16 WiMAX	...
	Physical	802.3 PHY	802.5 PHY	802.11 PHY	802.16 PHY	

TCP/IP – Revisited

❖ Encapsulation process for application data

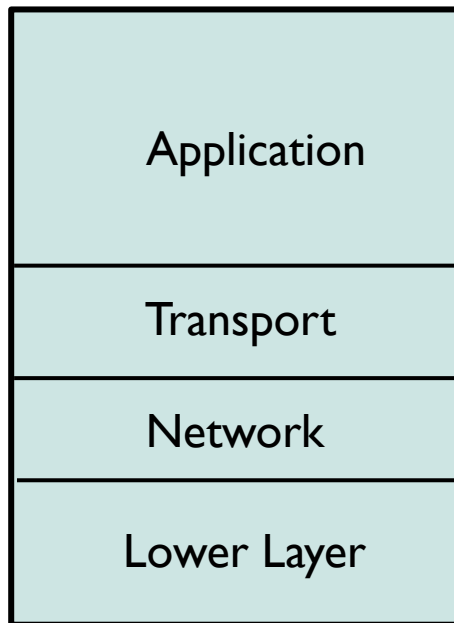
- each layer
 - takes a message from its upper layer, but
 - does not modify any bit of the message, and
 - appends additional information (header)



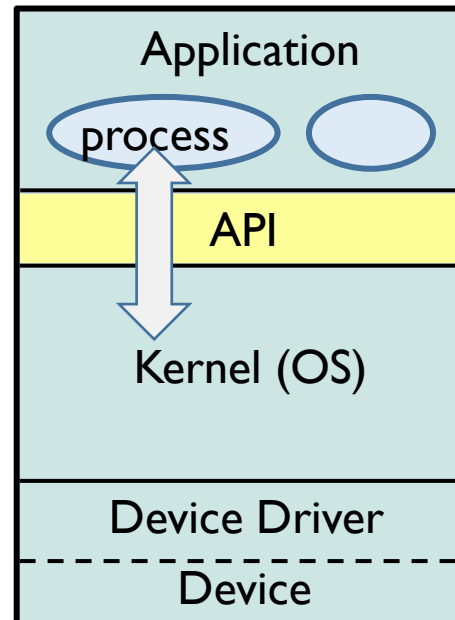
Where Sockets ?

❖ Sockets provide

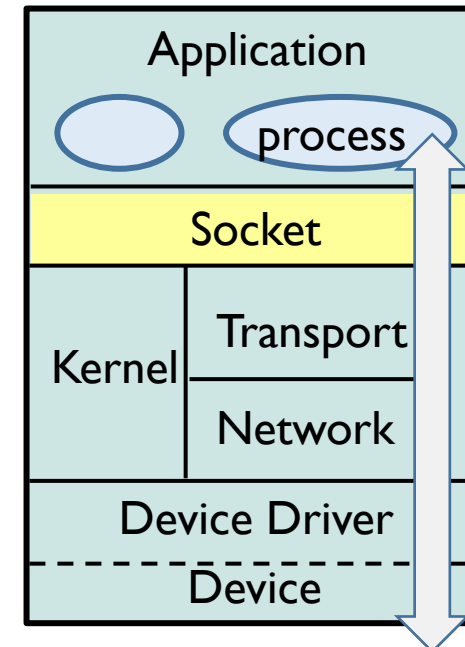
- network programming interface between
 - application process at application layer and
 - network protocols in kernel (OS)



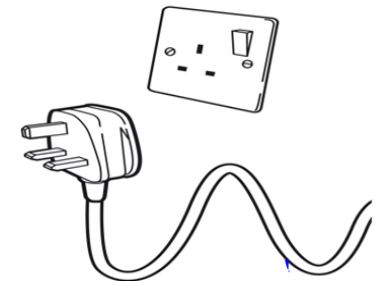
TCP/IP



OS

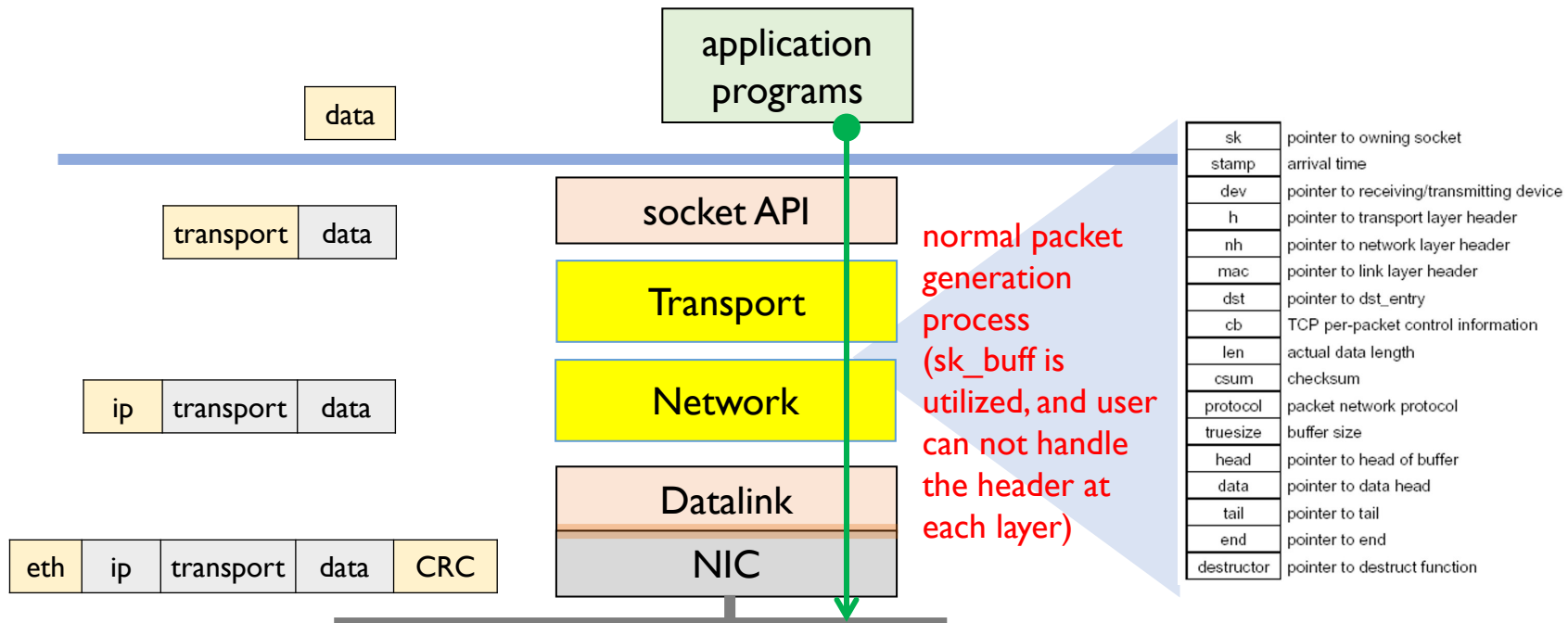


Network Programming



Standard Socket (1/3)

- ❖ In standard sockets,
 - the SDU from upper layer is encapsulated at a layer
 - TCP/UDP, IP, and Frame headers are automatically added, and
 - user don't need to aware the existence of TCP/IP layer

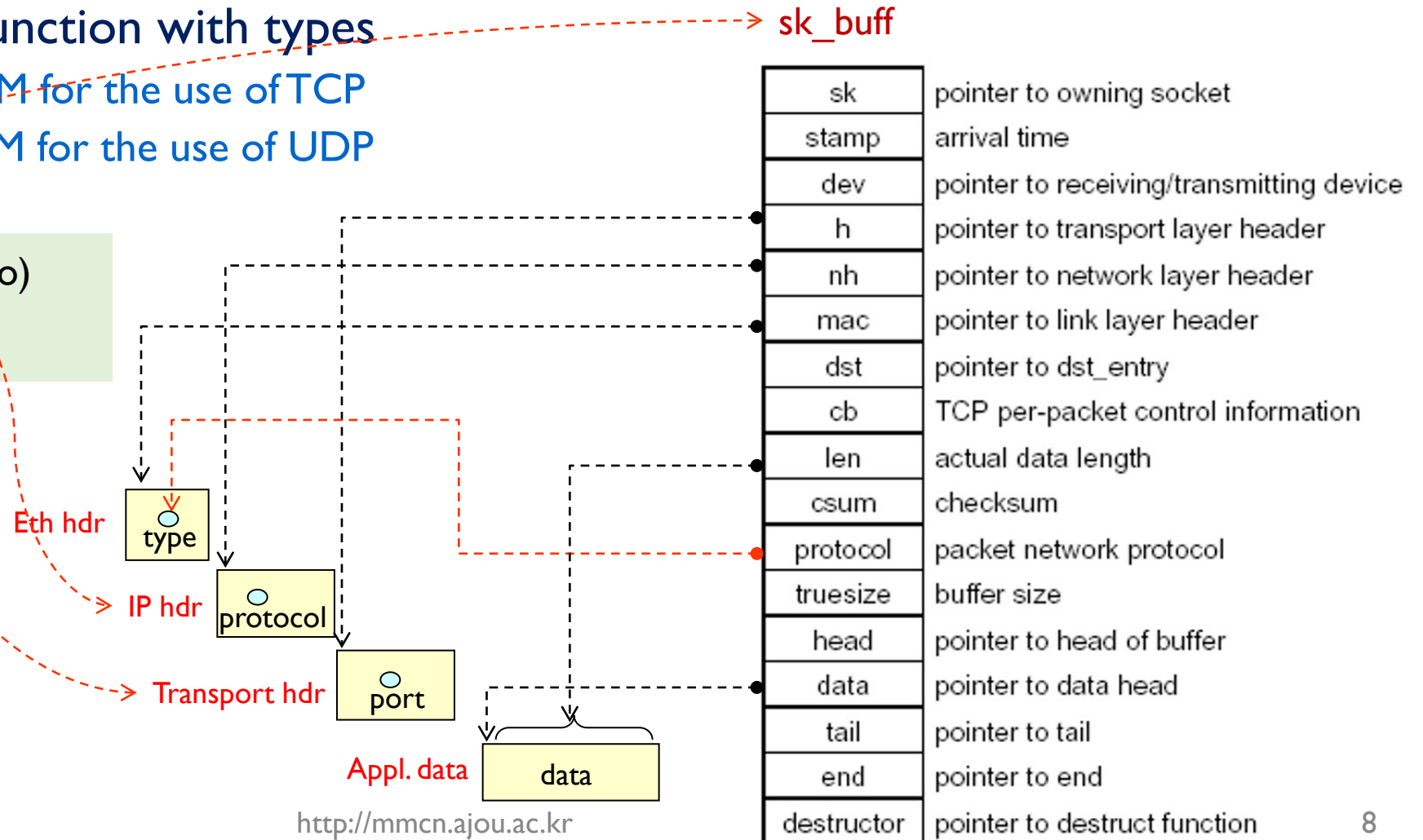


Standard Socket (2/3)

❖ To create standard sockets

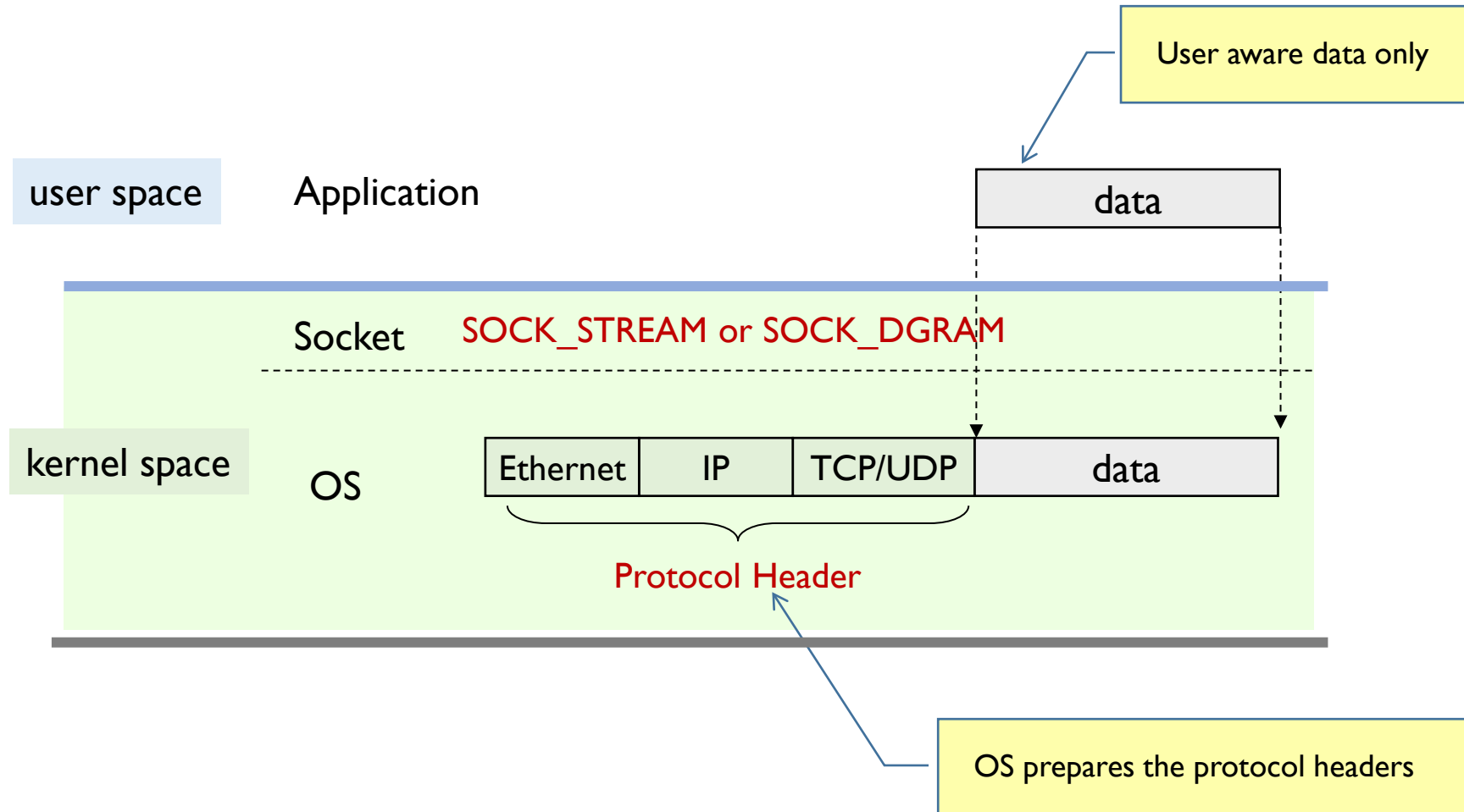
- use `socket( )` function with types
 - `SOCK_STREAM` for the use of TCP
 - `SOCK_DGRAM` for the use of UDP

```
sd = socket (dm, type, proto)
z = bind (sd, addr, len)
```



Standard Socket (3/3)

❖ SOCK_STREAM and SOCK_DGRAM

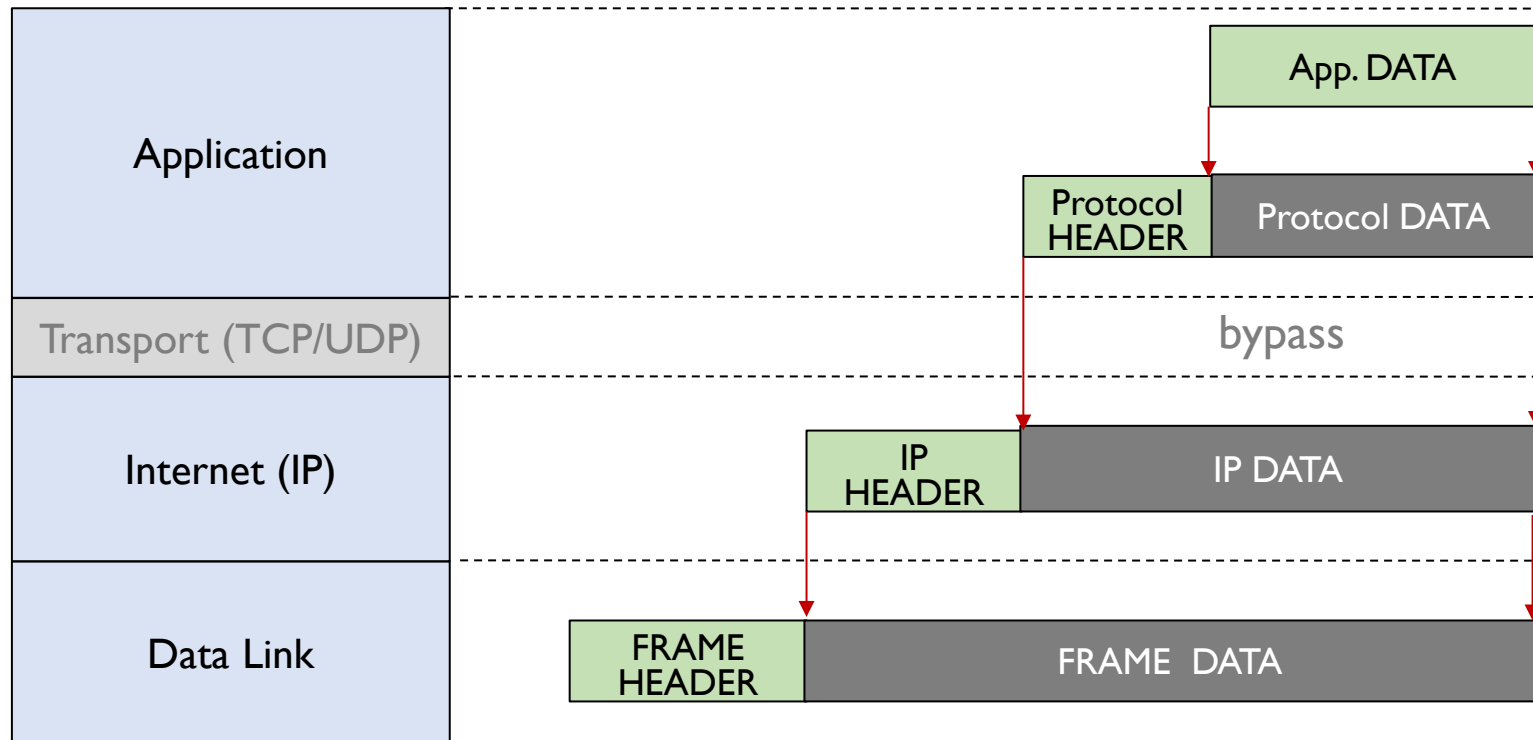




RAW SOCKETS

Raw Socket (1/3)

- ❖ A raw socket is
 - a type of socket that allows bypass the transport layer
 - user should prepare IP data part with appropriate headers and data



Raw Socket (2/3)

❖ To create raw sockets

- use `socket( )` function with type of `SOCK_RAW`

```
sockfd = socket (AF_INET, SOCK_RAW, proto) ;
```

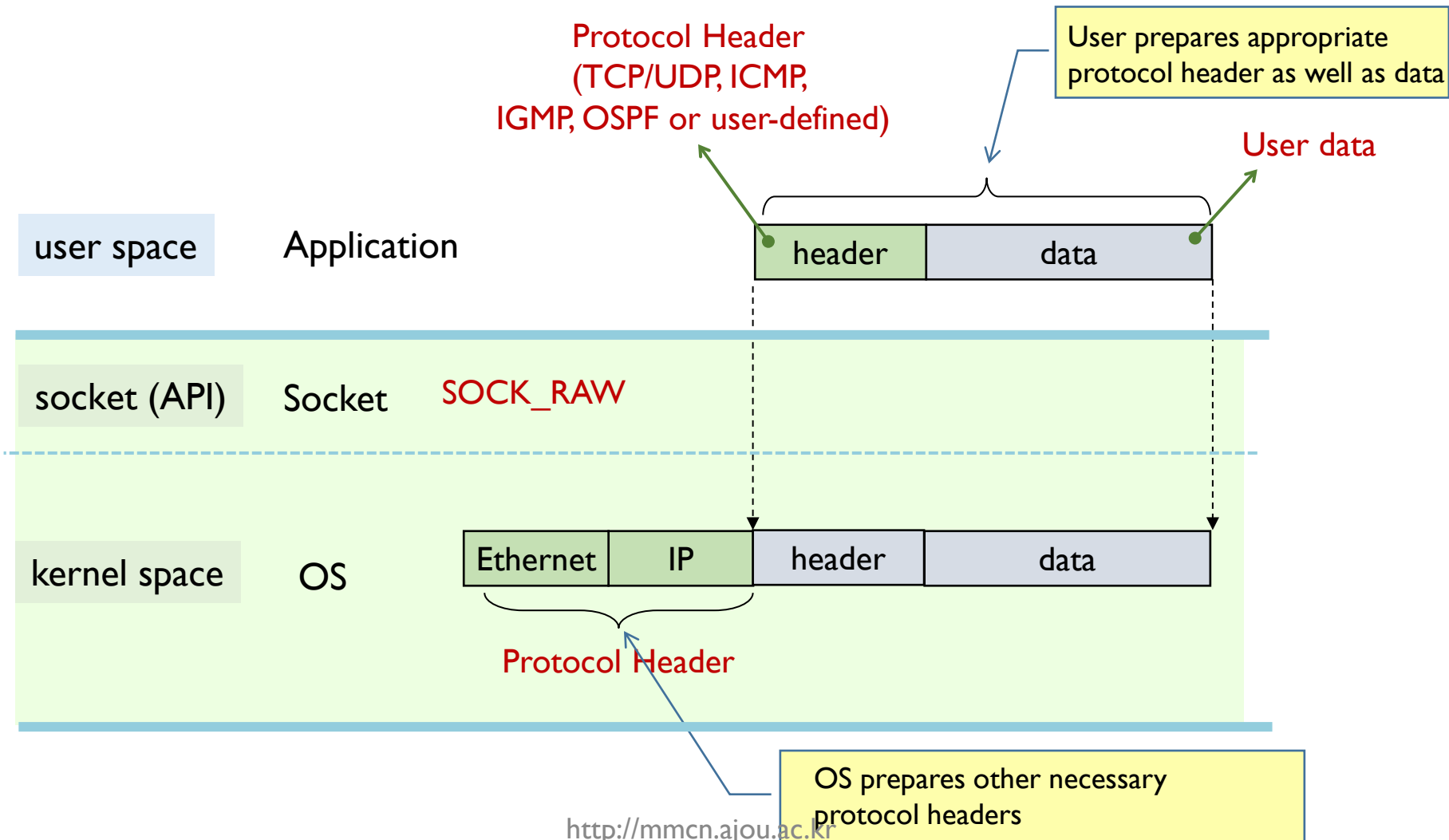
- where `proto` is `IPPROTO_RAW`, `IPPROTO_ICMP` etc.

❖ Typical usage of raw sockets

- Network layer protocols such as
 - ICMP (ping, traceroute, ...)
 - Routing protocols (OSPF, EIGRP, ...)
- Hacking or Attacks
 - generate TCP/UDP headers with faked headers

Raw Socket (3/3)

❖ RAW Socket



Configuration of RAW Socket

❖ SOCK_RAW

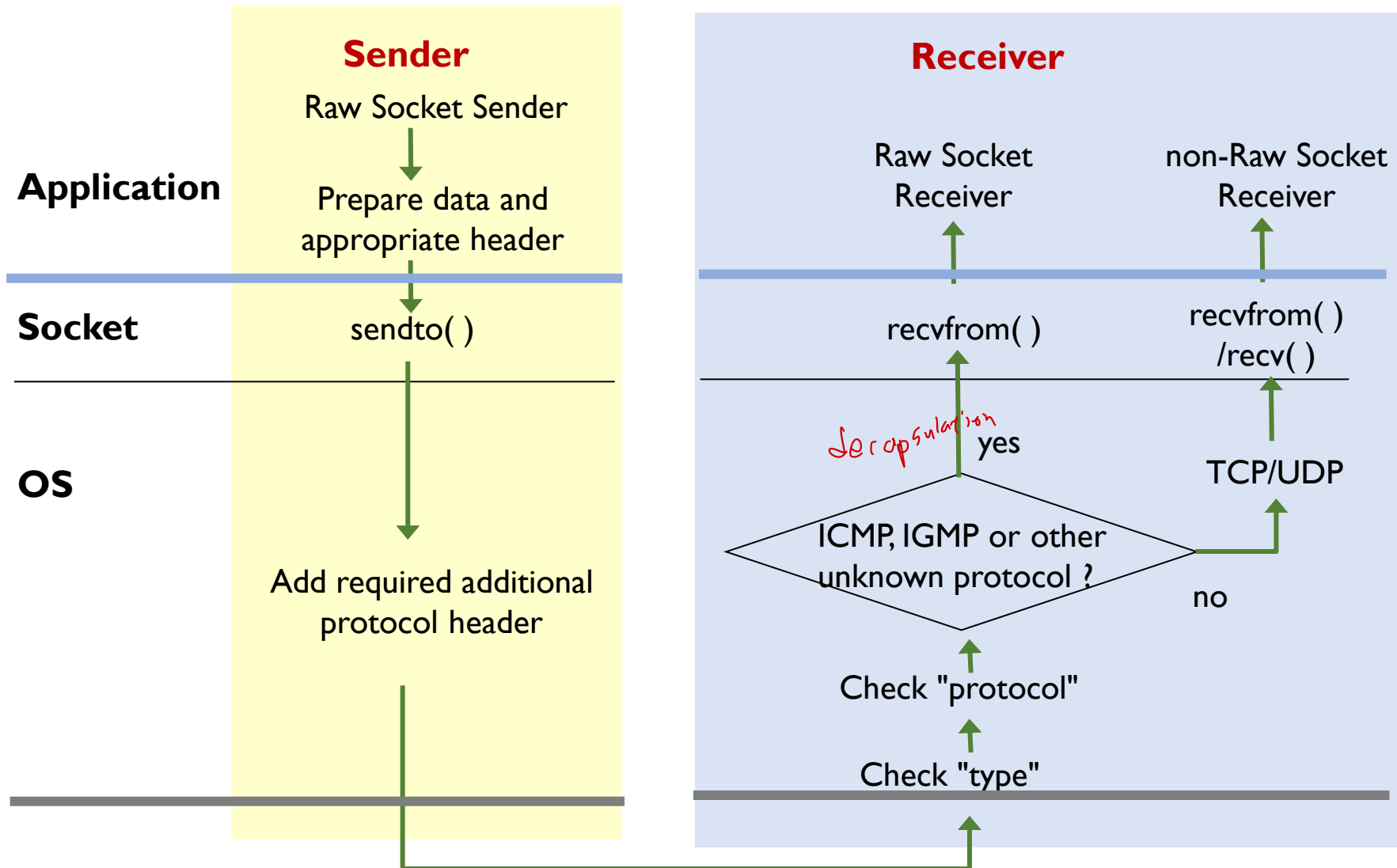
```
sockfd = socket (AF_INET, SOCK_RAW, proto) ;
```

protocol indicates that application will generate the protocol header

- protocol for RAW socket

Protocol	Header given by Application	Header by OS
IPPROTO_ICMP	ICMP (ICMP Header + ICMP Data)	Eth+IP
IPPROTO_IGMP	IGMP (IGMP Header + IGMP Data)	
IPPROTO_TCP	TCP (<u>TCP Header</u> + App. Data)	
IPPROTO_UDP	UDP (<u>UDP Header</u> + App. Data)	
user defined	user defined header (IP + USER)	

Raw Socket Sender & Receiver

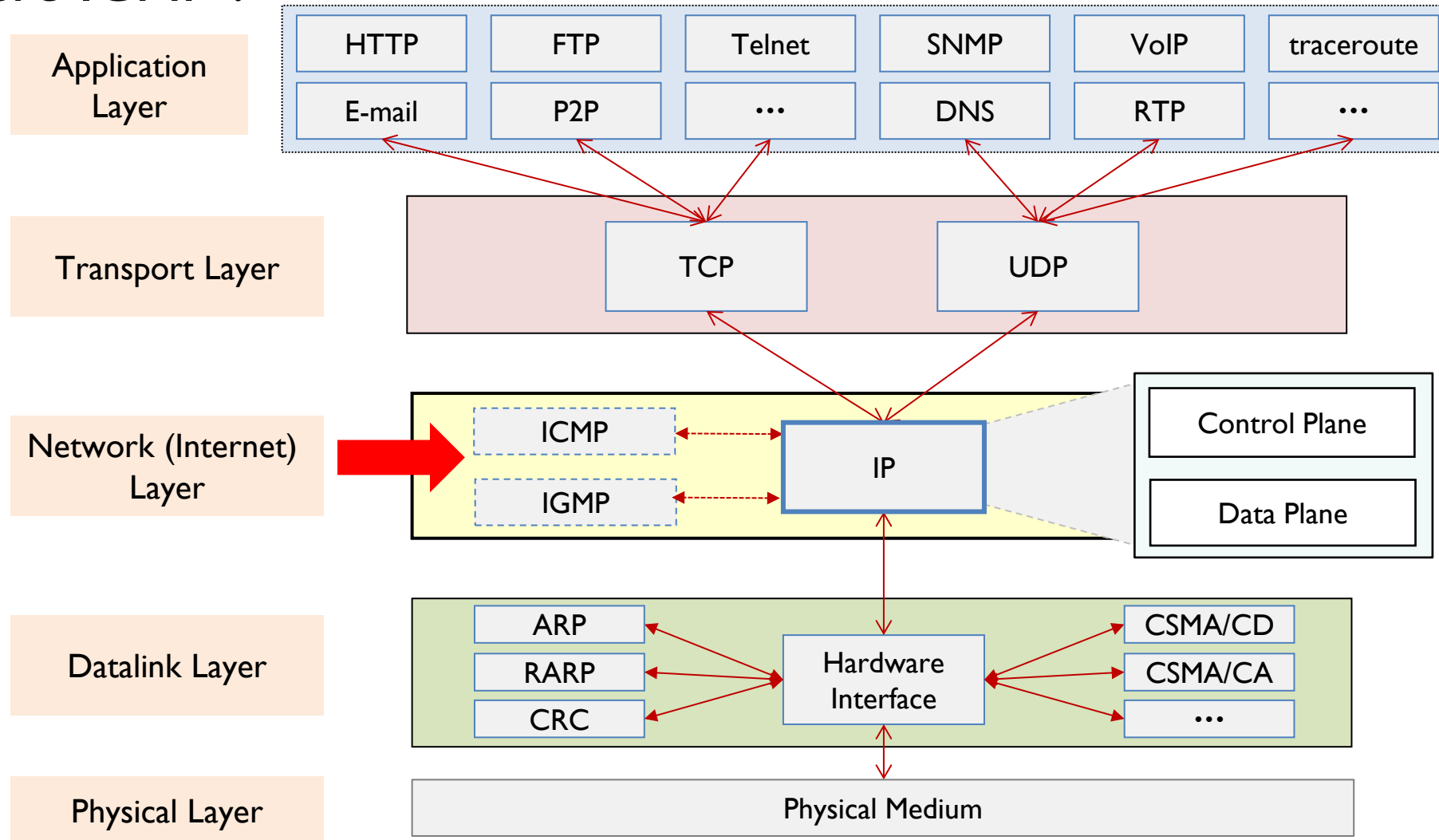




EXAMPLE: PING

TCP/IP Protocol Suite

❖ where ICMP ?

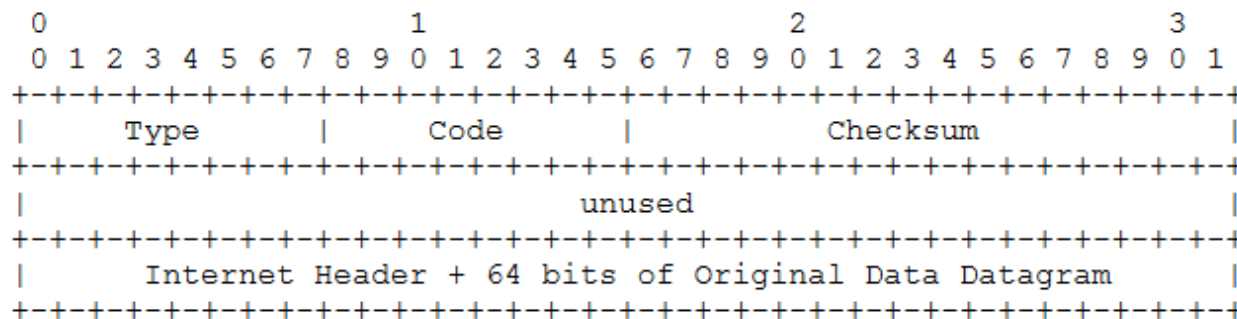


ICMP

- IP is unreliable 전송

❖ ICMP Overview

- ICMP is used by hosts and routers to communicate network-layer information to each other (RFC 792)
 - error reporting: unreachable host,
 - network, port, protocol
 - echo request/reply (used by ping)
- message format



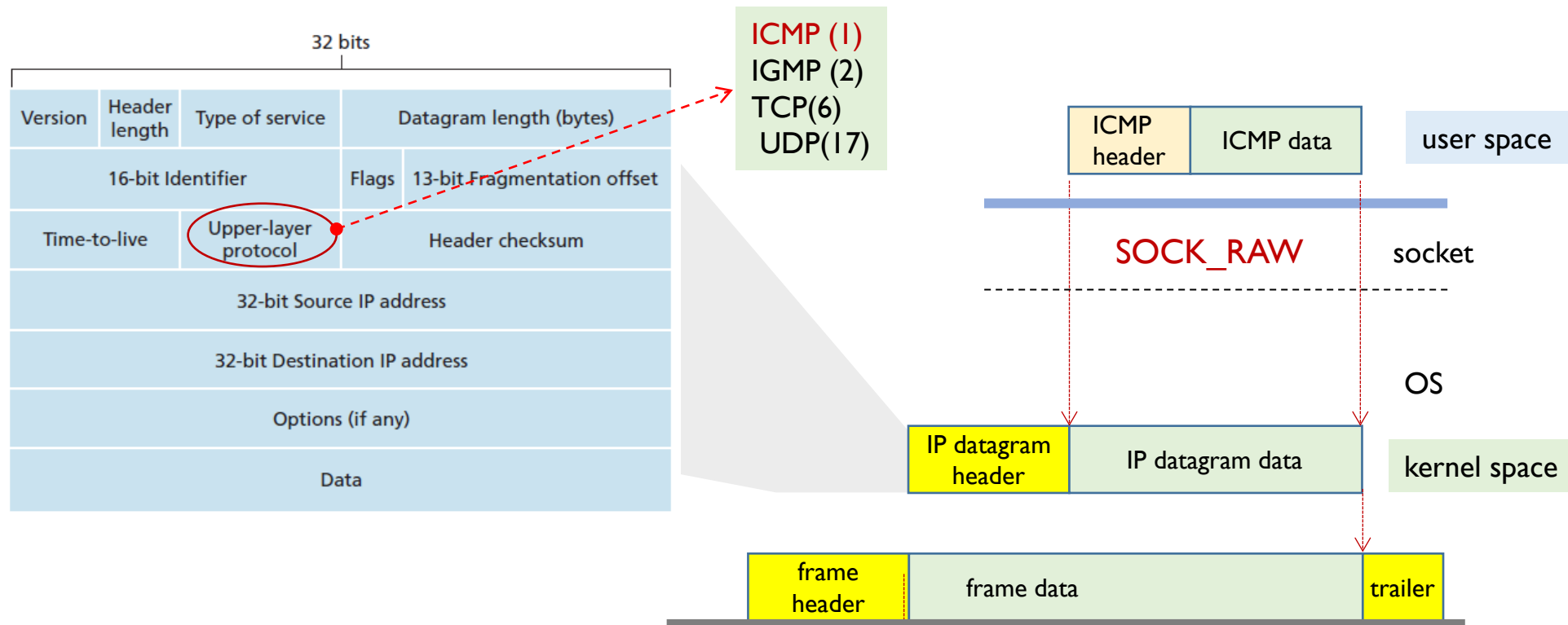
- Type : message type
- Code : information for message type
- Checksum : error detection for ICMP message only

Type	Code	Description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

ICMP

❖ ICMP message encapsulation

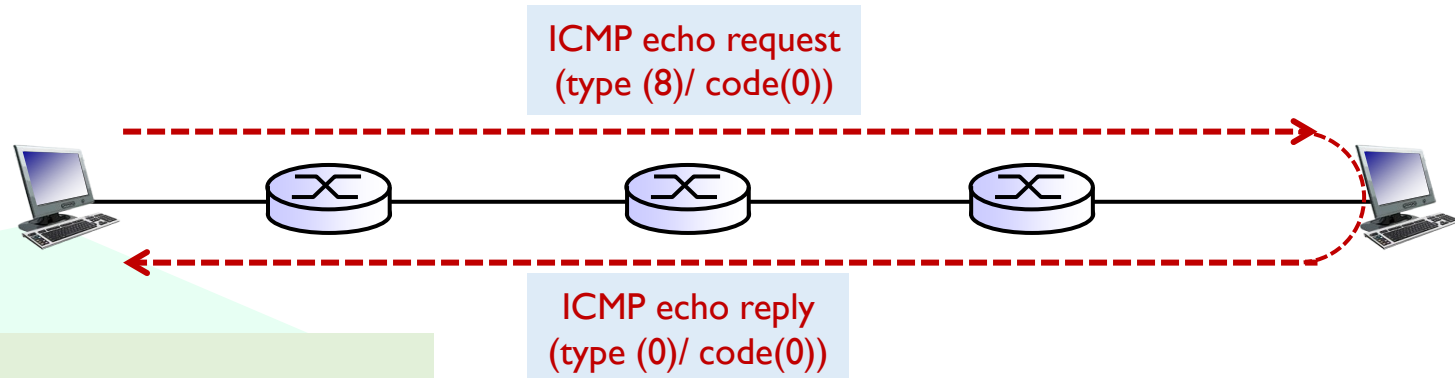
- Though ICMP message is encapsulated in IP datagram, ICMP is not an upper layer protocol over IP.
 - Instead, ICMP is a part of IP



ICMP – ping

❖ ping

- used to verify an end-to-end Internet path (or end host) is operational
- a part of ICMP (Internet Control Message Protocol)



```
$ ping -s 210.115.155.2
PING 210.115.155.2: 56 data bytes
64 bytes from 210.115.155.2: icmp_seq=0. time=56. ms
64 bytes from 210.115.155.2: icmp_seq=1. time=20. ms
64 bytes from 210.115.155.2: icmp_seq=3. time=34. ms
...
^C
----210.115.155.2 PING Statistics----
10 packets transmitted, 9 packets received, 10% packet loss
round-trip (ms)  min/avg/max = 20/39/76
$
```

Example #1 – raw_ping.c (1/5)

```
// some headers
...

#define BUFSIZE 1500
#define ICMP_ECHOREPLY 0
#define ICMP_ECHOREQUEST 8

// ICMP header
typedef struct _ICMPMESSAGE {
    u_char icmp_type; // type of message
    u_char icmp_code; // type sub code
    u_short icmp_cksum; // checksum
    u_short icmp_id; // identifier
    u_short icmp_seq; // sequence number
} ICMPMESSAGE;

// ICMP message decoding
void DecodeICMPMessage (char *buf, int bytes,
struct sockaddr_in *from);

// checksum calculation
u_short checksum (u_short *buffer, int size);
```

```
int main(int argc, char* argv[])
{
    int                sk, retval;
    char               *destIP;
    ICMPMESSAGE        icmpmsg;
    char               buf[BUFSIZE];
    struct sockaddr_in destaddr, peeraddr;
    int                addrlen, optlen;
    struct timeval      timeo = { 1, 0 };

    destIP = argv[1];

    // socket creation
    sk = socket (AF_INET, SOCK_RAW, IPPROTO_ICMP);

    // socket timeout options
    setsockopt (sk, SOL_SOCKET, SO_SNDTIMEO, &timeo, sizeof timeo);
    setsockopt (sk, SOL_SOCKET, SO_RCVTIMEO, &timeo, sizeof timeo);

    // socket for destination address
    memset(&destaddr, 0, sizeof(destaddr));
    destaddr.sin_family = AF_INET;
    destaddr.sin_addr.s_addr = inet_addr( destIP );
```

Example #1 – raw_ping.c (2/5)

```
for ( i=0; i<4; i++){
    // ICMP message initialization
    memset(&icmpmsg, 0, sizeof(icmpmsg));
    icmpmsg.icmp_type    = ICMP_ECHOREQUEST;
    icmpmsg.icmp_code    = 0;
    icmpmsg.icmp_id      = (u_short)getpid();
    icmpmsg.icmp_seq     = i;
    icmpmsg.icmp_cksum   = checksum (
                                (u_short *)&icmpmsg,
                                sizeof(icmpmsg));

    // send ICMP echo request message
    if ( sendto(  sk, (char *)&icmpmsg,
                sizeof(icmpmsg), 0,
                (struct sockaddr *)&destaddr,
                sizeof(destaddr)) < 0)
    {
        if( errno == EWOULDBLOCK ){
            printf("[error] Send timed out!\n");
            continue;
        }
        printf("sendto() error !!!\n");
        break;
    }
}
```

```
// ICMP echo reply report
addrlen = sizeof(peeraddr);
retval = recvfrom ( sk, buf, BUFSIZE, 0,
                    (struct sockaddr *)&peeraddr,
                    &addrlen));

if ( retval < 0 ) {
    if( errno == EWOULDBLOCK ){
        printf("[error] Request timed out!\n");
        continue;
    }
    printf("recvfrom() error !!!\n");
    break;
}

// ICMP echo reply decoding
DecodeICMPMessage(buf, retval, &peeraddr);

sleep(1);
} // end for

close(sk);
return 0;
} // end main
```

Example #1 – raw_ping.c (3/5)

```
void DecodeICMPMessage (char *buf, int len, struct sockaddr_in *from)
{
    IPHEADER *iphdr          = (IPHEADER *)buf;
    int iphdrlen              = iphdr->ip_hl * 4;
    ICMPMESSAGE *icmpmsg     = (ICMPMESSAGE *)(buf + iphdrlen);

    // error check
    if ( (len - iphdrlen) < 8 ) {
        printf("[error] ICMP packet is too short!\n");
        return;
    }

    if( icmpmsg->icmp_id != (u_short)getpid()){
        printf("[error] Not for our echo request!\n");
        return;
    }

    if( icmpmsg->icmp_type != ICMP_ECHOREPLY){
        printf("[error] Not a echo reply packet!\n");
        return;
    }

    // report
    printf("Reply from %s: total bytes = %d, seq = %d\n", inet_ntoa(from->sin_addr), len, icmpmsg->icmp_seq);
    return;
}
```

Example #1 – raw_ping.c (4/5)

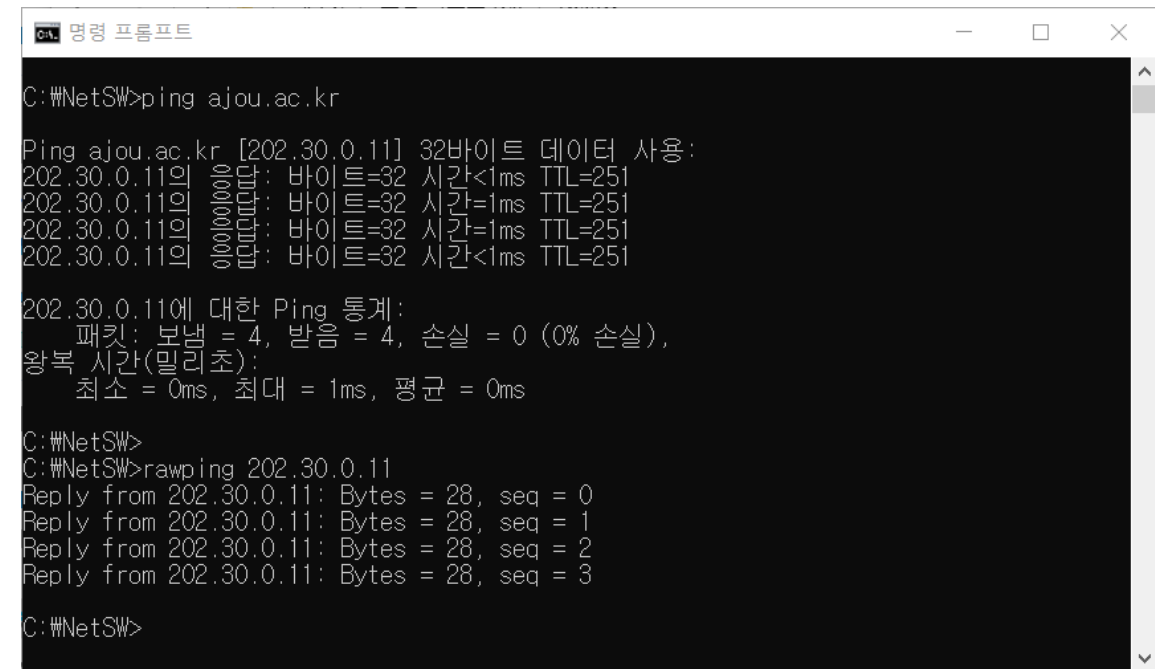
```
// checksum
u_short checksum (u_short *buf, int len)
{
    u_long cksum = 0;
    u_short *ptr = buf;
    int left = len;

    while(left > 1){
        cksum += *ptr++;
        left -= sizeof(u_short);
    }

    if(left == 1)
        cksum += *(u_char *)ptr;

    cksum = (cksum >> 16) + (cksum & 0xffff);
    cksum += (cksum >> 16);

    return (u_short)(~cksum);
}
```



The screenshot shows a Windows Command Prompt window titled "명령 프롬프트". The user has entered the command `C:\NetSW>ping ajou.ac.kr`. The output shows four successful ping responses from 202.30.0.11, each with 32 bytes of data, a time of less than 1ms, and a TTL of 251. Below this, the statistics for the ping are displayed: "202.30.0.11에 대한 Ping 통계: 패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실), 왕복 시간(밀리초): 최소 = 0ms, 최대 = 1ms, 평균 = 0ms". The user then enters `C:\NetSW>rawping 202.30.0.11`, and the output shows four successful raw ping responses from 202.30.0.11, each with 28 bytes of data and a sequence number (seq) of 0, 1, 2, and 3 respectively.

```
C:\NetSW>ping ajou.ac.kr

Ping ajou.ac.kr [202.30.0.11] 32바이트 데이터 사용:
202.30.0.11의 응답: 바이트=32 시간<1ms TTL=251
202.30.0.11의 응답: 바이트=32 시간=1ms TTL=251
202.30.0.11의 응답: 바이트=32 시간=1ms TTL=251
202.30.0.11의 응답: 바이트=32 시간<1ms TTL=251

202.30.0.11에 대한 Ping 통계:
    패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
왕복 시간(밀리초):
    최소 = 0ms, 최대 = 1ms, 평균 = 0ms

C:\NetSW>
C:\NetSW>rawping 202.30.0.11
Reply from 202.30.0.11: Bytes = 28, seq = 0
Reply from 202.30.0.11: Bytes = 28, seq = 1
Reply from 202.30.0.11: Bytes = 28, seq = 2
Reply from 202.30.0.11: Bytes = 28, seq = 3

C:\NetSW>
```

Example #1 – raw_ping.c (5/5)

ping

Capturing from 이더넷 (icmp)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Len	Info
1	0.000000	192.168.0.23	202.30.0.11	ICMP	74	Echo (ping) request id=0x0001, seq=113/28928, ttl=128 (reply...
2	0.000784	202.30.0.11	192.168.0.23	ICMP	74	Echo (ping) reply id=0x0001, seq=113/28928, ttl=128 (reply in 10)
3	1.003533	192.168.0.23	202.30.0.11	ICMP	74	Echo (ping) request id=0x0001, seq=114/29184, ttl=251 (request in ...)
4	1.004302	202.30.0.11	192.168.0.23	ICMP	74	Echo (ping) reply id=0x0001, seq=114/29184, ttl=251 (request in ...)
5	2.007022	192.168.0.23	202.30.0.11	ICMP	74	Echo (ping) request id=0x0001, seq=115/29440, ttl=128 (reply...
6	2.007856	202.30.0.11	192.168.0.23	ICMP	74	Echo (ping) reply id=0x0001, seq=115/29440, ttl=128 (reply in 10)
7	3.011668	192.168.0.23	202.30.0.11	ICMP	74	Echo (ping) request id=0x0001, seq=116/29696, ttl=128 (reply...

Protocol: ICMP (1)

Header checksum: 0x9e2f [validation disabled]
[Header checksum status: Unverified]
Source: 192.168.0.23
Destination: 202.30.0.11

Internet Control Message Protocol

Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0x4cea [correct]
[Checksum Status: Good]
Identifier (BE): 1 (0x0001)
Identifier (LE): 256 (0x0100)
Sequence number (BE): 113 (0x0071)
Sequence number (LE): 28928 (0x7100)

[Response frame: 2]

Data (32 bytes)

Data: 6162636465666768696a6b6c6d6e6f707172737475767761...
[Length: 32]

0020 00 0b 08 00 4c ea 00 01 00 71 61 62 63 64 65 66L...qabcdef
0030 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76ghijklmn opqrstuv

Data (data, data), 32 byte(s) Packets:

raw_ping

Capturing from 이더넷 (icmp)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Len	Info
4	1.004302	202.30.0.11	192.168.0.23	ICMP	74	Echo (ping) reply id=0x0001, seq=114/29184, ttl=251 (request in ...)
5	2.007022	192.168.0.23	202.30.0.11	ICMP	74	Echo (ping) request id=0x0001, seq=115/29440, ttl=128 (reply...
6	2.007856	202.30.0.11	192.168.0.23	ICMP	74	Echo (ping) reply id=0x0001, seq=115/29440, ttl=128 (reply in 10)
7	3.011668	192.168.0.23	202.30.0.11	ICMP	74	Echo (ping) request id=0x0001, seq=116/29696, ttl=128 (reply...
8	3.012569	202.30.0.11	192.168.0.23	ICMP	74	Echo (ping) reply id=0x0001, seq=116/29696, ttl=251 (request in ...)
9	17.045759	192.168.0.23	202.30.0.11	ICMP	42	Echo (ping) request id=0x6030, seq=0/0, ttl=128 (reply in 10)
10	17.046569	202.30.0.11	192.168.0.23	ICMP	60	Echo (ping) reply id=0x6030, seq=0/0, ttl=251 (request in ...)
11	18.048024	192.168.0.23	202.30.0.11	ICMP	42	Echo (ping) request id=0x6030, seq=256/1, ttl=128 (reply in ...)
12	18.048870	202.30.0.11	192.168.0.23	ICMP	60	Echo (ping) reply id=0x6030, seq=256/1, ttl=251 (request in ...)

Time to live: 128

Protocol: ICMP (1)

Header checksum: 0x9e4b [validation disabled]
[Header checksum status: Unverified]
Source: 192.168.0.23
Destination: 202.30.0.11

Internet Control Message Protocol

Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0x97cf [correct]
[Checksum Status: Good]
Identifier (BE): 24624 (0x6030)
Identifier (LE): 12384 (0x3060)
Sequence number (BE): 0 (0x0000)
Sequence number (LE): 0 (0x0000)

[Response frame: 10]

0010 00 1c 11 ad 00 00 80 01 9e 4b c0 a8 00 17 ca 1eK.....
0020 00 0b 08 00 97 cf 60 30 00 000..

Protocol (ip,proto), 1 byte(s) Packets: 16 · Displayed: 16 (100.0%) Profile: Default

ICMP – traceroute

❖ traceroute

- a computer network diagnostic tool for displaying the trace of the route (path) with packet delays
- implementation
 - by default, sending a sequence of UDP segments with TTL=1,2,...
 - ICMP echo request or TCP SYN segments can also be used

```
C:\W>tracert www.ietf.org

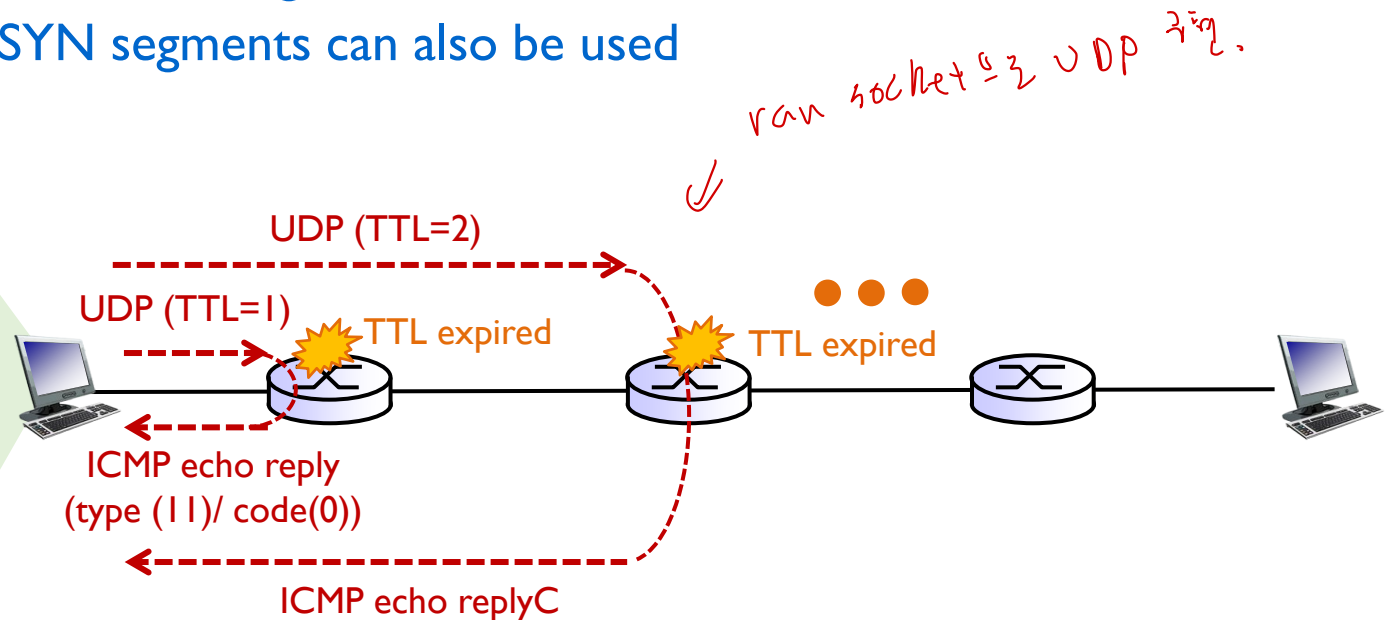
최대 30홉 이상의
www.ietf.org.cdn.cloudflare-dnssec.net [104.20.1.85]

  1   3 ms   2 ms   1 ms  172.30.1.254
  2   8 ms   5 ms   6 ms  211.35.140.254
  3   5 ms   2 ms   6 ms  220.85.110.37
  4  21 ms   3 ms   1 ms  61.78.42.167
  5   4 ms   3 ms   1 ms  112.189.28.1
  6   6 ms   2 ms   3 ms  112.174.17.237
  7   2 ms   3 ms   2 ms  112.174.34.134
  8   3 ms   3 ms   3 ms  128.134.40.130
  9   3 ms   2 ms   2 ms  104.20.1.85

추적을 완료했습니다.

C:\W>
```

3 probes





ICMP – traceroute

- ❖ Implementation ??
 - DIY !!!

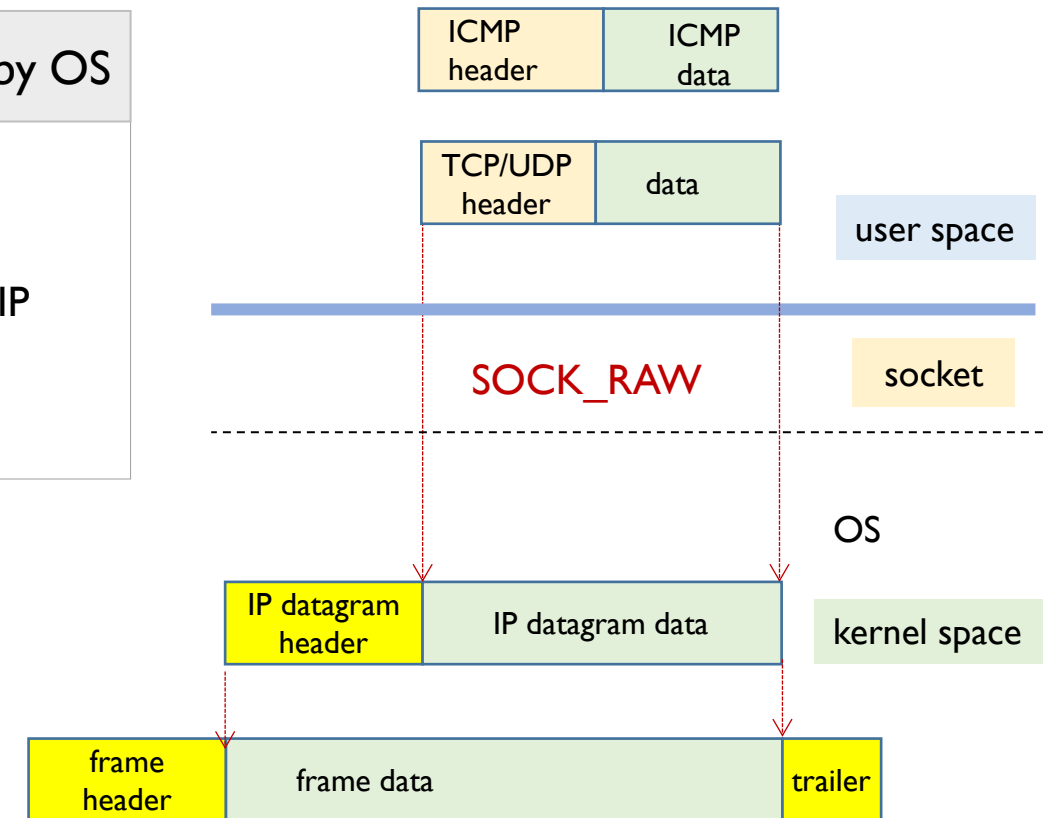


IP_HDRINCL OPTION

RAW Socket

- ❖ SOCK_RAW with no IP_HDRINCL option
 - protocol for RAW socket

Protocol	Header given by Application	Header by OS
IPPROTO_ICMP	ICMP (ICMP Header + ICMP Data)	Eth+IP
IPPROTO_IGMP	IGMP (IGMP Header + IGMP Data)	
IPPROTO_TCP	TCP (TCP Header + App. Data)	
IPPROTO_UDP	UDP (UDP Header + App. Data)	
user defined	user defined header (IP + USER)	



RAW Socket

❖ IP_HDRINCL

- Configured by `setsockopt()`
- It indicates the complete IP header is included
 - with the data and may be used only with the `SOCK_RAW` type.

❖ Configuring IP_HDRINCL option

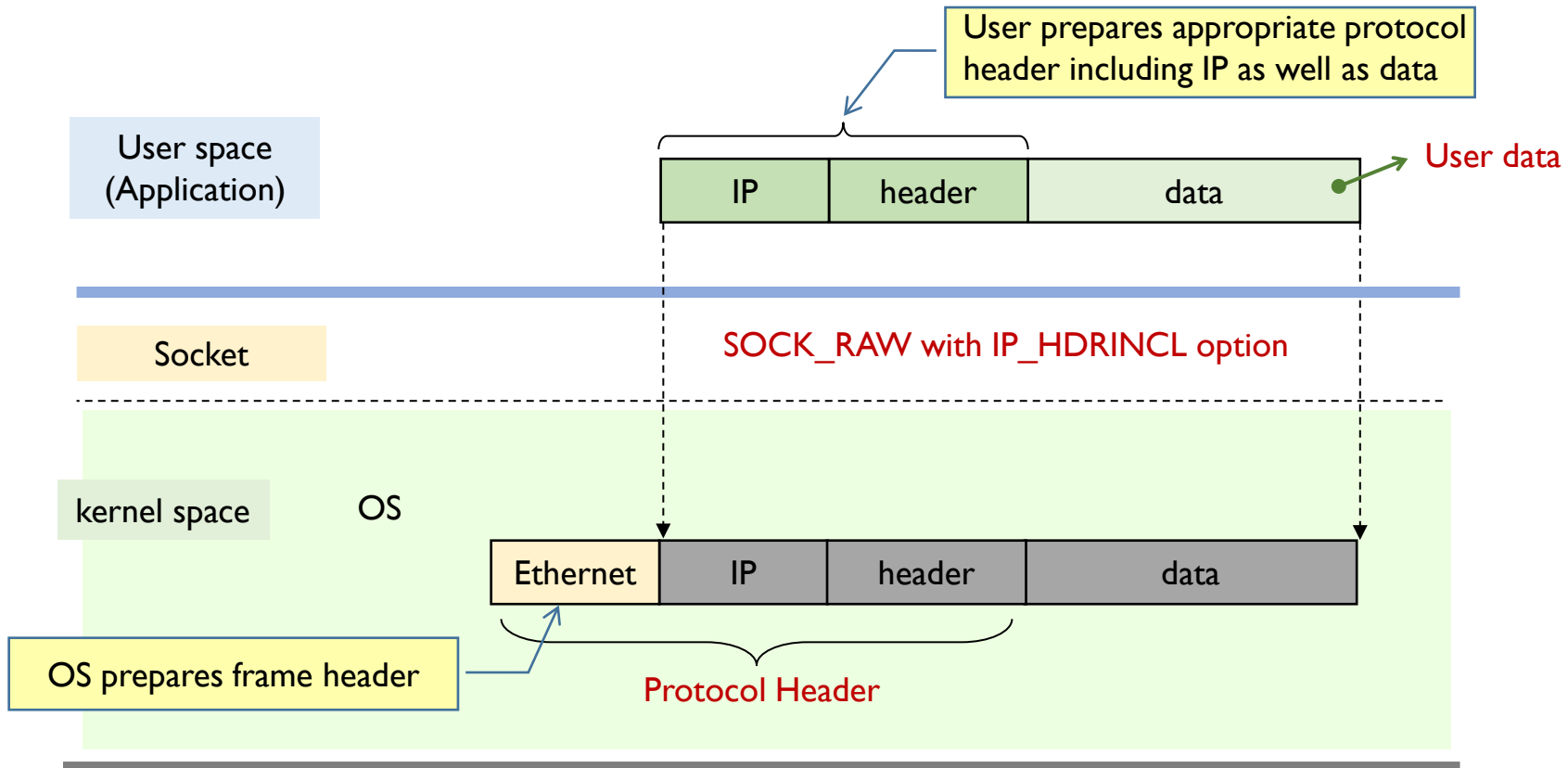
- Using `setsockopt()`

```
BOOL optval = TRUE;  
setsockopt ( s, IPPROTO_IP, IP_HDRINCL,  
            (char *)&optval, sizeof(optval));
```

IP_HDRINCL

~ IP header 3 with MAC 20701312.

❖ SOCK_RAW with IP_HDRINCL option





Refer to

- ❖ C Language Examples of IPv4 and IPv6 Raw Sockets for Linux
 - <http://www.pdbuchan.com/rawsock/rawsock.html>

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

